COMP3230 Principles of Operating Systems
Problem Set #1

Due date: 10:29 am, October 9, 2026

Total 6 points
(version: 1.0)

Note: This assignment weights a total of 6 points in the final marks of this course. However, just like each process has its own private virtual address space, this assignment also has its own virtual score space of 100 points, which is mapped to 6 points of physical marks. The below points for each question are given in the said virtual score space.

Question 1 – **OS Concept** (20 points)

(This question is related to ***ILO 1*** - "identify the core functions of the Operating Systems" and ***ILO 2a*** – "explain the principles behind the core functions – process/thread management".)

(a) (10%) One of the overheads induced by context switching is the occurrence of significant cache misses in the rescheduled process. A similar overhead can also arise during mode switching. Why does mode switching have cache misses?

(b) (10%) Suppose an operating system reserves one CPU core exclusively for executing kernel code, while the remaining cores handle user-space application processes. Would this approach reduce the cache-related overhead mentioned above? Would it be beneficial overall? Justify your answer.

Question 2 – **Mode/Context Switch** (25 points)

(This question is related to ***ILO 1*** - "identify the core functions of the Operating Systems" and ***ILO 2a*** – "explain the principles behind the core functions – process/thread management".)

Consider a single-core system with two processes, P and Q, and the following behavior of the two processes.

- P is currently executing in user mode.
- Q is in the Ready state.
- P executes *read(fd, buf, 4)*.
- The user-level read() function is implemented by a library wrapper of the syscall sys_read().
- The file descriptor fd refers to a blocking input source. At this moment, no data is available.
- If the read cannot complete immediately, the operating system dispatches Q to run.
- After a while, the device receives data and raises an interrupt.
- The interrupt handler processes the event and wakes P. The scheduler dispatches P to run immediately.

(a) (15%) Please choose the proper process (P or Q), mode (user or kernel) and switch (mode or context) in the following execution history involved in the above procedure.

P/Q user/kernel mode
- syscall
- mode/context switch

P/Q user/kernel mode
- executing sys_read
- no data is available; process becomes blocked.
- mode/context switch

P/Q user/kernel mode
- return-from-trap
- mode/context switch

P/Q user/kernel mode
- device interrupt
- mode/context switch

P/Q user/kernel mode
- executing the interrupt handler
- mode/context switch

P/Q user/kernel mode
- resuming the original sys_read
- return-from-syscall
- mode/context switch

P/Q user/kernel mode
- continuing at the code following the syscall instruction inside the read() library wrapper

(b) Answer the following questions:

1. (5%) If data is already available when P executes read(), is a context switch required even though Q is in the Ready state? Explain your answer.
2. (5%) If a device interrupt wakes P but the scheduler decides to let Q continue running, which switch(es) will occur at that point, and what state is P in?

Question 3 – Process Control (25 points)

(This question is related to ***ILO2*a** – "explain the principles behind the core function: Process Management", and ***ILO 4*** – "demonstrate knowledge in applying system software and tools".)

A Unix-like shell runs this sequence:

1. Shell process `S` calls `fork()` to create child `C`.
2. Child `C` calls `execve("/bin/echo", ["echo","hi"], envp)`.
3. Parent `S` calls `waitpid(C, &status, 0)`.
4. `echo` prints and exits with status `0`.

Assume no errors. Complete the following tasks:

(a) (10%) After `fork()` and before `execve()`, identify 2 properties that parent and child share logically and 3 properties that are distinct.
(b) (10%) After `execve()`, specify exactly what (at least 3 items) is replaced in `C` and what (at least 2 items) remains associated with the same process identity.
(c) (5%) If parent `S` never calls `waitpid`, what observable process-abstraction artifact appears in the OS?

Question 4 - Process Scheduling (30 points)

(This question is related to ***ILO 2a*** - "discuss the mechanisms and policies in efficiently sharing of CPU resources" and ***ILO 3*** – "analyze and evaluate the algorithm and explain the performance issues".)

(a) Consider the following scenario in a single-CPU system with two processes, P1 and P2:

- P1 is an interactive process that alternates between CPU and I/O bursts. It runs on the CPU for 1 ms, then performs I/O for 5 ms. This cycle repeats five times, followed by a final CPU burst of 1 ms. The total CPU time required by P1 is therefore 6 ms.
- P2 is a CPU-bound process that runs continuously on the CPU for 30 ms without any I/O.
- At t = 0, both P1 and P2 are ready and P1 is ahead of P2 in the queue.

i. (8%) Calculate the total turnaround time for P1 when scheduled using Round Robin (RR) with a time quantum of 4 ms.

ii. (8%) Calculate the total turnaround time for P1 when scheduled using a Multilevel Feedback Queue (MLFQ) scheduler with three queues:

- Queue 1 (highest priority): Time quantum = 2 ms
- Queue 2 (medium priority): Time quantum = 4 ms
- Queue 3 (lowest priority): Time quantum = 10 ms

Demotion rule: If a process uses its entire time quantum, it is demoted to the next lower priority queue. Assume no priority boost.

iii. (4%) Based on your calculations, which scheduler provides better service to P1? Justify your answer.

(b) (10%) Describe how a lottery scheduling algorithm could be made to approximate a CPU scheduling algorithm that always runs the job that hasn't run in the longest time.